

As a Senior Backend Software Architect, I have designed the **AI Fitness Coach** architecture to prioritize high availability, modularity, and low-latency AI inference.

1. Architecture Overview

We will utilize a **Microservices-oriented architecture** deployed on **AWS/GCP** using **Docker & Kubernetes (EKS/GKE)**.

- **API Gateway:** Nginx or Kong to handle rate limiting, request routing, and SSL termination.
- **Async Processing:** RabbitMQ or AWS SQS for decoupling the AI generation task from the user request cycle.
- **Database:** PostgreSQL (Primary) for relational data and Redis for caching and session management.

2. Services

- **Auth Service:** Manages JWT issuance and user identity.
- **Profile/Onboarding Service:** Manages biometric data and equipment preferences.
- **Plan Service:** Orchestrates AI calls and parses structured outputs (JSON/Markdown).
- **Tracking/Analytics Service:** Aggregates logs and measurement trends for dashboarding.
- **Notification Service:** Scheduled worker (Cron) to trigger email/push alerts.

3. Modules

- **AI Engine Adapter:** Abstracted module to swap AI providers (OpenAI, Anthropic) without breaking business logic.
- **Constraint Validator:** Middleware ensuring business rules (e.g., 24hr measure limits) are enforced before data reaches the persistence layer.

4. Authentication

- **JWT (JSON Web Tokens):** Short-lived access tokens (15m) + secure HTTP-only refresh tokens stored in database.
- **Provider:** Use Auth0 or AWS Cognito for standard compliance (GDPR) and secure OIDC workflows.

5. Authorization

- **RBAC (Role-Based Access Control):** Simple system for `USER` vs `ADMIN`.
- **Resource Ownership:** Middleware checking `user\_id` on every database query (row-level security/tenant isolation).

6. Business Logic

- **Domain-Driven Design (DDD):** Logic is encapsulated in the Services layer.
- **AI Orchestration:** Using a **"Queue-Wait-Poll"** pattern for plan generation to stay within the 5s performance requirement. If the model takes > 2s, we return a `202 Accepted` status with a Job ID.

7. Background Jobs

- **Tool:** `BullMQ` (Node.js) or `Celery` (Python).
- **Use Cases:** Weekly progress report generation, pushing motivational tips, and batch-triggering AI plan regenerations.

8. File Storage

- **AWS S3:** Only for storing user profile images (if added later) or exported data reports. Using signed URLs for secure access.

9. Error Handling

- **Global Exception Middleware:** Catch-all for API errors, logging trace IDs, and returning masked errors to users (never leak stack traces).
- **Circuit Breakers:** Implement in the AI Engine Adapter. If the AI provider is down, fallback to a "Default Template" strategy to maintain service.

10. Logging

- **Structured Logging:** ELK Stack (Elasticsearch, Logstash, Kibana) or Datadog for centralized log management and tracing.
- **Audit Trails:** Mandatory logging of any changes to user measurements or plans.

11. Folder Structure (Example: Node/TypeScript)

```
/src
  /api           # Controllers/Routes
  /config       # Environment/DB connection
```

```
/services      # Business logic (PlanService, Auth)
/models        # DB Schema definitions (TypeORM/Prisma)
/jobs          # BullMQ/Worker tasks
/middleware    # Auth, Logging, Validation
/utils         # AI Client, Date manipulators
/tests         # Unit/Integration tests
```

12. Recommended Libraries

- **Backend:** Node.js (TypeScript) or Go for performance.
- **ORM:** Prisma (Excellent type safety for PostgreSQL).
- **API Documentation:** Swagger/OpenAPI.
- **Testing:** Jest (Unit), Supertest (Integration).
- **Validation:** Joi or Zod (Input schema validation).
- **AI Integration:** LangChain (to abstract prompts, memory, and model switching).

Scaling Strategy (The 10k+ User Challenge)

1. **Database Read Replicas:** Route analytical queries (charts) to a read-only replica to keep the primary node free for writes.
2. **Redis Caching:** Cache the "Current Active Plan" to prevent redundant AI calls on every dashboard refresh.
3. **Horizontal Autoscaling:** Configure HPA (Horizontal Pod Autoscaler) in Kubernetes to spin up more pods when CPU usage exceeds 70%.